Exact pole transport of the actual norm membership remainder

2026-11-14. Lean source freeze `23d78212afe6b379e51040abf66622305250f62b`. All claims unsigned [pcc:unverified].

The actual norm support and the full membership remainder now have exact polynomial transport identities under the inverse-pair condition `e*l=1`. They apply at every Dickson depth and imply an exact coefficient normalization. The proof uses polynomial composition, norm division and monic remainder uniqueness; it does not substitute agreement on finite-field values for polynomial equality. Main unsigned claims: remainder transport `PCC-20261114-d754b9` and coefficient transport `PCC-20261114-9e7aa6` [pcc:unverified].

Lean source, prospective contract, exact declaration map, editable diagram, SVG, report PDF.

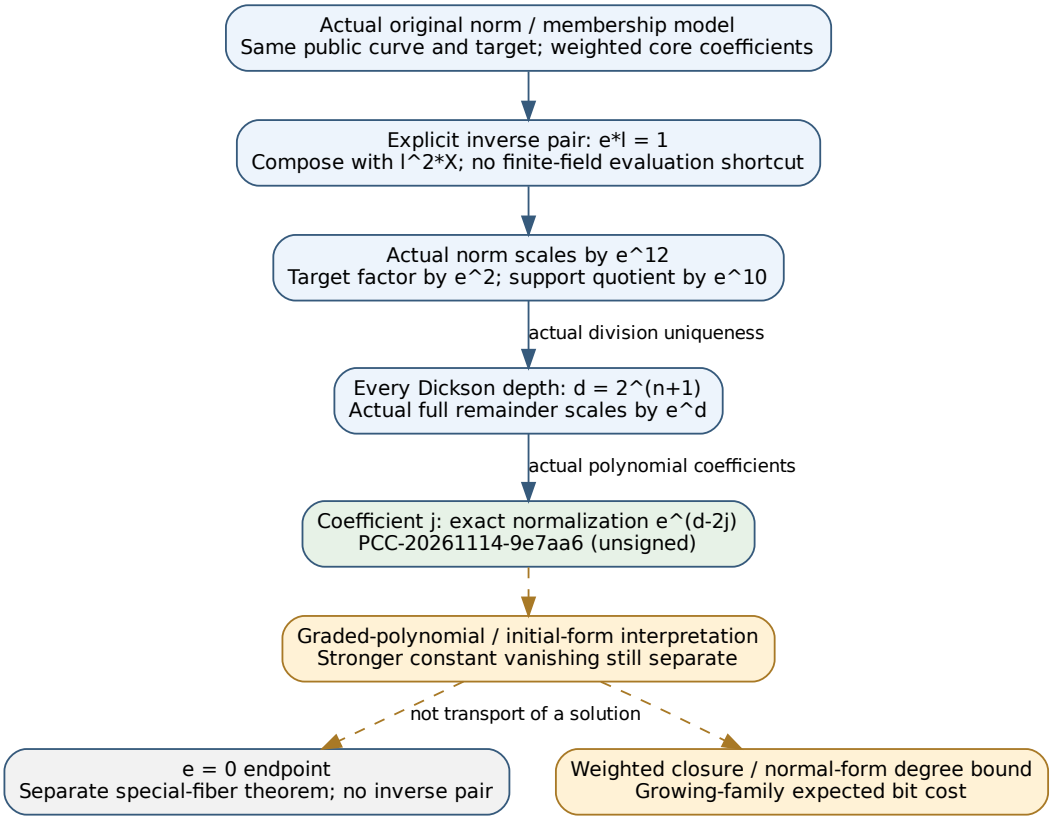


Figure 1: Exact invertible-parameter transport and the separate graded/solver obligations. Solid arrows are source-bound Lean identities; dashed arrows remain open. All evidence unsigned.

Actual objects and the exact map

Over a field K, the original public target satisfies `z^2=r^3+a*r+b`. The scalar and elementary polynomial identities need only a commutative ring and an explicit inverse pair. The actual source chart uses

```
D = X^3+a2*X^2+a1*X;  B = c0+c1*X
A = completeTarget(D,B,r,z) = D + z*B(r)-D(r)
N = A^2-(X^3+a*X+b)*B^2
U = N div (X-r)
P_n = master(2,n)-terminal;  R_n = P_n mod U.
```

Target completion and the curve equation make norm division exact. The actual support is monic degree5. These are

constructed polynomials, with no successful coefficient, divisor witness or solve oracle supplied. The scalar functions are lifted through the actual constant-polynomial homomorphism to actual univariate polynomials [pcc:unverified].

Set `e*l=1`. Keep the original curve and target on the source side, replace its four retained coefficients by

`(a1*l^4, a2*l^2, c0*l^3, c1*l),  and compose with l^2*X.`

The completed A normalizes by `e^6`, B by `e^3`, and the cubic by `e^6`. Consequently the actual norm normalizes by `e^12`. The target factor normalizes by `e^2`, so the actual quotient normalizes by `e^10`:

```
e^10 * U_original_scaled_core(l^2*X) = U_e.
```

Here `U_e` is exactly the existing `NormWeightedLeading.scaledSupport`, with curve `X^3+e^4*a*X+e^6*b` and target `(e^2*r,e^3*z)`. The quotient equation is derived from the actual norm identity and monic division uniqueness, not assumed as a transport premise in the main theorem. For j<=5, the coefficient equation has normalization `e^(10-2*j)` [pcc:unverified].

At every depth n let `d=2^(n+1)`. The actual Dickson master-minus-terminal normalizes by `e^d` to the existing scaled master-minus-scaled-terminal. The full remainder then obeys the polynomial identity

```
e^d * R_original_scaled_core(l^2*X) = R_e,n.
```

For `2*j<=d`, taking coefficients gives

```
e^(d-2*j) * (R_original_scaled_core).coeff(j) = R_e,n.coeff(j).
```

At depth4 the four nonconstant residuals have nominal normalization exponents24,26,28,30 in descending X order. These are exact exponents in the transport equation, not a measured degree profile or a solver bound. The nominal constant exponent is32; its earlier predicted actual weight30 requires a further graded vanishing/divisibility argument [pcc:unverified].

Why the remainder proof is substantive

The generic remainder lemma assumes two monic degree5 moduli and the exact unit/variable scaling relation between them. Its proof transports the actual quotient/remainder equation. For any nonnegative normalization exponent d, the new quotient is the old composed quotient multiplied by `e^d*l^10`; this handles small depths without a negative exponent. The transported remainder stays below degree5 by the actual composition/scalar degree bounds. Monic division uniqueness identifies it with the computed remainder. The actual norm support theorem discharges the modulus-scaling premise in the main membership theorem [pcc:unverified].

For field-valued transport, the inverse pair excludes e=0. The previous component separately computes that endpoint and proves its four coefficient equations force the origin. No theorem transports the special-fiber tuple to an original solution. At that endpoint the curve equation is `y^2=x^3`; it is used for coefficient analysis, without an elliptic-group transport assertion. The proof covers general prime fields through its field hypotheses; no special curve, subgroup order or hidden integer label is assumed [pcc:unverified].

Validation and construction boundaries

The module contains23 new statements and retains build receipts001–005, including the failed full-tower drafts003–005, and final successful build006. Only standard foundations occur in its axiom prints. The whole-study gate reports327 statements,0 additional trusted uses, exit0; import-inclusive fresh replay also exits0. Unsigned validation claim `PCC-20261114-5a59a9` binds the receipt, exact source/artifact hashes and preserved failures. The canonical adapter has22 passing tests; format/build/issuance pass, with4 inherited unchanged SHA256 strict-clippy style failures. All25 new source/data claims remain unsigned; default-PATH verdicts are `not_covered` (exit2). Dependency pins remain Lean4.35.0, CryptoLean revision `148f5dc389b341401e06d76abe7783c4ccf42f2e`, and Mathlib revision `cfc55fc58714a59dc06e6e79edc6d1d95140d3c2`. These are the preceding component's unchanged dependency revisions [pcc:unverified].

The separate canonical identity driver seals only this unconditional intermediate ring step:

```
e^12*(A^2-F*B^2) = (e^6*A)^2-(e^6*F)*(e^3*B)^2.
```

Certificate `IDC1h504690879b070928`, unsigned claim `PCC-20261114-c8beeb`, has syntactic degree15 and32 statement-derived plus16 extra replay points. It does not encode the inverse-pair condition or the polynomial/remainder transport,

and supplies no signed execution evidence. The unchanged canonical adapter's checks and its inherited SHA256 strict-clippy failures are retained separately [pcc:unverified].

No native fixture experiment, random sample, root/branch search, F4/msolve, paid/cloud/attack run or performance comparison was attempted here. Flow for the earlier fixed native fixture experiment remains1 planned,0 attempted, 0 completed,0 excluded,1 admission-blocked by checkpoint upload. This component is universal mathematical proof/certificate construction. The old benchrec1680/162 diagnostic remains untouched, along with all earlier source, evidence, claims, ledger bytes and compiler failures [pcc:unverified].

Sources checked and remaining obligations

Sources checked (KB): same unavailable tools/wrapper as already recorded; no unchanged failed retry. Reuse the bounded prior review, the actual old norm/scaledSupport/scaledMaster definitions, and pinned Mathlib Polynomial.Eval.Degree `comp_C_mul_X_coeff`, composition identities, Polynomial.Div monic uniqueness, and the actual degree lemmas used in source. These standard operations are reused without a novelty/absence claim [pcc:unverified].

The next proof is the graded-polynomial interpretation of these exact invertible identities, including any stronger constant-coefficient vanishing. An initial-ideal equality and a weighted normal-form/iterated-closure bound remain open. Establishing those bounds for a growing family and pricing finite-field extraction, yield, relation collection and all expected bit costs remains necessary for the broader goal. Input normalization alone does not supply such a bound [pcc:unverified].

The original W-jet family still needs its own public-input map. This proof does not identify it with the norm-membership family. Global performance scoreboard/timeline and curve graphs remain unchanged: no measured speed or certified group transport edge was added. The earlier study-local initial- form diagram also remains valid: the exact invertible scaling map here strengthens its algebraic foundation while its graded/solver arrows remain unresolved [pcc:unverified].

Exact proof map

Exact hypotheses and source binding hashes are in claim-pins.json. Every claim is unsigned [pcc:unverified].

NormPoleTransport declaration	Unsigned claim
`inversePowers`	`PCC-20261114-f5b06b`
`completedValueScale`	`PCC-20261114-55d16a`
`branchValueScale`	`PCC-20261114-367118`
`curveValueScale`	`PCC-20261114-00cb72`
`normValueScale`	`PCC-20261114-9a26b0`
`completedValuePolynomial`	`PCC-20261114-e99a16`
`actualAComp`	`PCC-20261114-4b131c`
`actualAPoleScale`	`PCC-20261114-16801b`
`branchPoleScale`	`PCC-20261114-eb3429`
`curvePoleScale`	`PCC-20261114-255b47`
`normPoleScale`	`PCC-20261114-136a79`
`targetFactorPoleScale`	`PCC-20261114-f5a15d`
`masterPoleScale`	`PCC-20261114-6857b8`
`terminalPoleScale`	`PCC-20261114-cabd66`
`actualSupportExact`	`PCC-20261114-769bd6`
`supportPoleScale`	`PCC-20261114-97225a`
`supportCoefficientScale`	`PCC-20261114-351787`
`supportNormalizedCoefficient`	`PCC-20261114-955e02`
`actualSupportWindow`	`PCC-20261114-4fec47`
`scaledSupportWindow`	`PCC-20261114-88a657`
`remainderPoleScale`	`PCC-20261114-0750e7`
`membershipPoleScale`	`PCC-20261114-d754b9`
`membershipCoefficientScale`	`PCC-20261114-9e7aa6`